

Module 8

Quick Reference and Course Summary

Learning objectives

- Provide a single quick reference and course summary for Verilog and SystemVerilog across IEEE versi...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Version Timeline (One Table)"]

T1 --> N["Next module in course"]

Provide a single quick reference and course summary for Verilog and SystemVerilog across IEEE versions (1364-1995 through 1800-2017).

Overview

- This module is a ****reference and summary**** for the version-centric Verilog/SystemVerilog course (Mo...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

M1["Module 1-3 Verilog"] --> M4["Module 4-6 SV design"]

M4 --> M7["Module 7 compare/migrate"]

M7 --> M8["Module 8 quick ref"]

M8 --> PROJ["Your project README standard"]

Module 8: DUT structure and key interfaces (see `common_dut/rtl/`).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

REF["Module 8 tables"] --> PLAN["test plan: which moduleN.sh"]

PLAN --> RUN["run owning module scripts"]

M7T["Module 7 equiv tests"] --> RUN

Module 8: UVM layers, agents, and checkers for this phase.

1. Course knowledge architecture (not a new DUT)

- Module 8 is a meta-layer: maps Modules 1–7 to standards, constructs, and learning path
- No new RTL — architecture here is how reference material is organized, not a chip block
- Sections: version timeline → construct-by-standard → design subset → migration → tool support
- Runnable examples (``module8/examples/``): each prints one slice of the quick reference for terminal...

2. How design subsets relate across the course

- Verilog-only path: Modules 1–3 (1364-1995 → 2001 → 2005) — ``wire`/`reg``, ``always @*``
- SystemVerilog design path: Modules 4–6 — ``logic``, ``always_comb`/`always_ff``, interfaces, packages
- Integration path: Module 7 side-by-side pairs;
Module 8 tables tie constructs to modules
- Choosing a subset: project README should state revision + allowed construct list from Module 8 tabl...

3. Reference-to-RTL traceability

- `construct_lookup` example: answers “which module introduced X?” — links study back to Modules 1–6 DU...
- `course_map` example: module titles map to ``docs/MODULEn.md`` and ``moduleN/dut/`` where applicable
- `synthesizable_subset` example: do/avoid rules apply to all prior DUT architectures

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

LOOKUP["construct_lookup"] --> REV["pick IEEE revision"]

REV --> SUB["pick design subset"]

SUB --> CI["CI flags match subset"]

CI --> REG["regression module1-7.sh"]

Module 8: how tests, checks, and metrics close verification goals.

1. Verification strategy without new testbenches

- Module 8 does not add tests — it documents how to test when applying course knowledge
- Regression rule: after picking a standard from Module 8 tables, run the owning module's ``./scripts/...`
- Migration rule: use Module 7 equivalence tests whenever syntax changes under stable behavior
- Subset rule: CI/lint flags should match the documented Verilog-only vs SV-design subset

2. Lookup-driven test planning

- Before using a construct, run `construct_lookup` /
Section 2 table — note minimum module and revision
- pitfalls example: common mistakes (wrong standard, mixed subsets) become test-plan checklist items
- `version_selection` example: document
simulator/synthesis flags next to chosen revision in
project RE...

3. Closure and ongoing maintenance

- learning_path example: ordered study 1→6, then 7;
use 8 during real projects as guardrails
- When a new IEEE revision ships: add Module N, update
Module 7 pairs and Module 8 tables — re-run al...
- Golden rule: Module 8 guides **what** to run; Modules
1–7 provide the actual tests and DUTs

Syllabus topics

1. Version Timeline (One Table)

- See docs/MODULE8.md

Hands-on examples

Module 8 self-check

```
$ ./scripts/module8.sh --check
```

Terminal: module8_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Demo: Construct Lookup

```
$ cd module8/examples/construct_lookup && make clean && make run
```

Terminal: ex01_construct_lookup

```
rm -f top
iverilog -g2012 -o top construct_lookup.sv
vvp top
=== Module 8: Construct lookup (first standard) ===
logic      -> 1800-2005
always_comb/always_ff -> 1800-2005
interface, modport -> 1800-2005
package, import  -> 1800-2005
unique/priority case -> 1800-2005
always @*        -> 1364-2001
ANSI ports       -> 1364-2001
generate         -> 1364-2001
signed           -> 1364-2001
localparam       -> 1364-2001
defparam         -> 1364-1995 (deprecated in 1364-2005)
Checker          -> 1800-2012
Unified LRM      -> 1800-2017
Full table: module8/docs/version_table.md
=== End construct lookup ===
construct_lookup.sv:24: $finish called at 0 (1s)
```

Demo: Course Map

```
$ cd module8/examples/course_map && make clean && make run
```

Terminal: ex02_course_map

```
rm -f top
iverilog -g2012 -o top course_map.sv
vvp top
=== Module 8: Course map (Modules 1-8) ===
 1 | IEEE 1364-1995 | Verilog-95: wire/reg, assign, always/initial, explicit sensitivity
 2 | IEEE 1364-2001 | ANSI ports, always @*, generate, signed, arrays, localparam
 3 | IEEE 1364-2005 | Clarifications, synthesizable subset, defparam deprecated
 4 | IEEE 1800-2005 | logic, always_comb/always_ff, interfaces, packages, unique/priority ca
 5 | IEEE 1800-2009/12 | Interface refinements, checkers, array/string methods, assertions
 6 | IEEE 1800-2017 | Unified LRM, Verilog as subset of 1800, current standard
 7 | Version comparison | Side-by-side, migration patterns, tool support, checklists
 8 | Quick reference | This module: tables, cheat sheet, course map
Learning path: 1->2->3->4->5->6 (version order); then 7; use 8 as reference
Links: module8/docs/course_map.md
=== End course map ===
course_map.sv:20: $finish called at 0 (1s)
```

Demo: Design Subset

```
$ cd module8/examples/design_subset && make clean && make run
```

Terminal: ex03_design_subset

```
rm -f top
iverilog -g2012 -o top design_subset.v
vvp top
=== Module 8: Design subset quick reference ===
Verilog-only (1364):
  Ports: ANSI (2001+) or non-ANSI (1995)
  Types: wire, reg (no logic)
  Comb: always @* or always @(inputs)
  Seq: always @(posedge clk) with <=
  No: logic, always_comb/always_ff, interfaces, packages, unique/priority case
SystemVerilog design (1800):
  Ports: ANSI; logic typical
  Types: logic (single driver)
  Comb: always_comb
  Seq: always_ff with <=
  Connectivity: interface+modport; package+import; unique/priority case
Use 1364 when: tools or IP require Verilog only
Use 1800 when: flow supports 1800; preferred for new RTL
=== End design subset ===
design_subset.v:24: $finish called at 0 (1s)
```

Demo: Learning Path

```
$ cd module8/examples/learning_path && make clean && make run
```

Terminal: ex04_learning_path

```
rm -f top
iverilog -g2012 -o top learning_path.v
vvp top
=== Module 8: Learning path and next steps ===
Learning path: 1 -> 2 -> 3 -> 4 -> 5 -> 6 (version order)
Then: Module 7 (comparison and migration); use Module 8 as reference anytime
Where to go next:
  Project: Apply version selection and migration (Module 7); use Module 8 as quick reference
  LRM: IEEE 1800-2017 for authoritative syntax and semantics
  Tools: Simulator, synthesis, lint manuals for supported revision and synthesizable subset
  Related: RTL design patterns, UVM, SVA, tool-specific training
  New revisions: Add new module; update Module 7 and Module 8 tables
=== End learning path ===
learning_path.v:18: $finish called at 0 (1s)
```


Demo: Migration Steps

```
$ cd module8/examples/migration_steps && make clean && make run
```

Terminal: ex05_migration_steps

```
rm -f top
iverilog -g2012 -o top migration_steps.v
vvp top
=== Module 8: Migration steps quick reference ===
1364-1995 -> 1364-2001:
  1. Ports -> ANSI  2. Comb -> always @*  3. generate/signed/localparam if needed  4. Test
1364-2001/2005 -> 1800 (design subset):
  1. wire/reg -> logic  2. always @* -> always_comb  3. posedge clk -> always_ff
  4. Optional: interfaces, packages, unique/priority case  5. Remove defparam  6. Test
1800-2005 -> 1800-2009/2012/2017:
  1. Run regression (usually compatible)  2. Adopt new features if supported  3. Set project s
Rules: one block/feature at a time; keep external interface unchanged; one standard, document
Full checklists: docs/MODULE7.md
=== End migration steps ===
migration_steps.v:19: $finish called at 0 (1s)
```

Demo: Pitfalls

```
$ cd module8/examples/pitfalls && make clean && make run
```

Terminal: ex06_pitfalls

```
rm -f top
iverilog -g2012 -o top pitfalls.v
vvp top
=== Module 8: Common pitfalls (quick reference) ===
 1. Assuming a construct is in an older standard -> Use construct-by-standard table (Section 2)
 2. Mixing subsets without a rule -> Define one project subset and revision; document in style
 3. Skipping regression after migration -> Run sim/synth after every step; use Module 7 checkli
 4. Forgetting tool support -> Match project standard to tool-supported revision and synthesiza
 5. Treating Module 8 as replacement for 1-7 -> Use Module 8 for quick lookup; refer to Modules
    Full pitfalls: docs/MODULE8.md Common Pitfalls and How to Avoid Them
=== End pitfalls ===
pitfalls.v:16: $finish called at 0 (1s)
```

Demo: Synthesizable Subset

```
$ cd module8/examples/synthesizable_subset && make clean && make run
```

Terminal: ex07_synthesizable_subset

```
rm -f top
iverilog -g2012 -o top synthesizable_subset.sv
vvp top
=== Module 8: Synthesizable subset (both 1364 and 1800) ===
Do: assign, always_comb or always @*, always_ff or always @(posedge clk)
Do: generate, parameter, localparam, single driver per net
Avoid: Delays (#), initial (except FPGA init per tool), fork/join
Avoid: system tasks in RTL, defparam, multiple drivers (unless tri-state), unintended latches
Check: Tool manual for exact synthesizable subset and revision
=== End synthesizable subset ===
synthesizable_subset.sv:15: $finish called at 0 (1s)
```

Demo: Tool Support

```
$ cd module8/examples/tool_support && make clean && make run
```

Terminal: ex08_tool_support

```
rm -f top
iverilog -g2012 -o top tool_support.sv
vvp top
=== Module 8: Tool support summary ===
Simulators: 1364-2001/2005; 1800 subset varies by tool
Synthesis: Synthesizable subset of 1364 and/or 1800; revision varies
Lint:      1364 and 1800; often selectable revision
Formal:    Assertions/checkers (1800); revision varies
Icarus:    Primarily 1364-2001/2005; limited SystemVerilog
Verilator: 1364 and growing 1800 subset
Commercial (VCS, Questa, Xcelium): Typically 1800-2005 through 1800-2017
Match project standard to simulator, synthesis, and IP support.
=== End tool support ===
tool_support.sv:18: $finish called at 0 (1s)
```

Demo: Version Selection

```
$ cd module8/examples/version_selection && make clean && make run
```

Terminal: ex09_version_selection

```
rm -f top
iverilog -g2012 -o top version_selection.sv
vvp top
=== Module 8: Version selection quick reference ===
Match: project standard to simulator, synthesis, and IP support
State: one revision (e.g. IEEE 1800-2017) and subset (e.g. design only, no SVA in RTL)
Document: in style guide or README; reference LRM and tool manuals
Verilog-only: use when tools or IP require 1364; 1364-2005 as reference
SystemVerilog design: use when flow supports 1800; 1800-2017 as reference
Full checklist: docs/MODULE7.md (version-selection checklist)
=== End version selection ===
version_selection.sv:16: $finish called at 0 (1s)
```

Demo: Version Timeline

```
$ cd module8/examples/version_timeline && make clean && make run
```

Terminal: ex10_version_timeline

```
rm -f top
iverilog -g2012 -o top version_timeline.sv
vvp top
=== Module 8: Version timeline ===
1364-1995 | 1995 | First Verilog; base language      | Module 1
1364-2001 | 2001 | ANSI, @*, generate, signed      | Module 2
1364-2005 | 2005 | Last Verilog-only; defparam deprecated | Module 3
1800-2005 | 2005 | First SystemVerilog; logic, always_comb/ff | Module 4
1800-2009 | 2009 | Interface refinements; clarifications | Module 5
1800-2012 | 2012 | Checker; array/string methods      | Module 5
1800-2017 | 2017 | Unified LRM; current standard      | Module 6
Comparison and migration      | Module 7
Quick reference and summary    | Module 8
Path: 1->2->3->4->5->6 (version order); 7 (comparison); 8 (reference)
=== End version timeline ===
version_timeline.sv:20: $finish called at 0 (1s)
```

Practice & assessment

Exercises

- Lookup
- Subset
- Migration
- Course map
- Cheat sheet

Exercises

- Assuming a construct is in an older standard
- Mixing subsets without a rule
- Skipping regression after migration
- Forgetting tool support
- Treating this module as a replacement for Modules 1-7

Summary & next steps

- Provide a single quick reference and course summary for Verilog and SystemVerilog across IEEE versi...
- Complete module8/CHECKLIST.md
- Run `./scripts/module8.sh --check`
- Next: Next module in course